

Informe de Pruebas Unitarias

Cordillera Platform — Microservicios Backend

Proyecto: Cordillera Platform

Equipo: Grupo Cordillera

Integrantes: Ignacio Valeria, Benjamín Palma, Benjamín Flores

Herramienta: JaCoCo / JUnit 5

Fecha: Junio 2026

1. Introducción

Este documento registra las pruebas unitarias y de integración que hicimos sobre los cuatro microservicios backend de la Cordillera Platform: BFF Gateway, Data Service, KPI Service y Report Service.

Para medir cobertura usamos JaCoCo integrado al build de Maven. Configuramos un *quality gate* que corta el build si la cobertura de instrucciones baja del 60%. Todos los servicios lo pasan.

2. Resultados de Cobertura

Microservicio	Herramienta	Mínimo	Instrucciones	Branches	Estado
BFF Gateway	JaCoCo / JUnit 5	60%	88%	70%	PASS
Data Service	JaCoCo / JUnit 5	60%	100%	100%	PASS
KPI Service	JaCoCo / JUnit 5	60%	61%	72%	PASS
Report Service	JaCoCo / JUnit 5	60%	86%	76%	PASS

El Data Service es el único con cobertura completa (100% instrucciones y ramas). El KPI Service queda justo en el límite en instrucciones (61%), pero lo supera en ramas (72%). El desglose por paquete explica por qué: los calculadores y el cliente alcanzan 100%, mientras que el paquete de configuración se queda en 4% por clases de infraestructura que Spring carga en contexto.

2.1 BFF Gateway

bff-gateway

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
cl.duoc.cordillera.bffgateway.auth.service		66%		66%	5	14	19	48	3	11	0	1
cl.duoc.cordillera.bffgateway.service		95%		83%	13	65	4	179	4	32	0	3
cl.duoc.cordillera.bffgateway.dto		75%		50%	20	31	9	47	9	20	0	1
cl.duoc.cordillera.bffgateway.auth.controller		37%		n/a	3	5	4	8	3	5	0	1
cl.duoc.cordillera.bffgateway.config		86%		0%	3	11	4	23	1	9	0	5
cl.duoc.cordillera.bffgateway.exception		74%		n/a	2	9	6	23	2	9	0	4
cl.duoc.cordillera.bffgateway.controller		98%		50%	2	36	1	77	0	34	0	5
cl.duoc.cordillera.bffgateway		37%		n/a	1	2	2	3	1	2	0	1
Total	234 of 2.121	88%	30 of 102	70%	49	173	49	408	23	122	0	21

Figura 1. Cobertura JaCoCo — BFF Gateway (88% instrucciones, 70% ramas)

El paquete `service` llega al 95% de instrucciones y al 83% de ramas. El `controller` está al 98%. Lo que baja el total son los paquetes `auth.controller` (37%) y el paquete raíz (37%), ambos con muy pocas líneas.

2.2 Data Service

data-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
cl.duoc.cordillera.dataservice.config		100%		100%	0	4	0	26	0	3	0	2
cl.duoc.cordillera.dataservice.service		100%		n/a	0	9	0	18	0	9	0	1
cl.duoc.cordillera.dataservice.controller		100%		n/a	0	8	0	11	0	8	0	1
cl.duoc.cordillera.dataservice.exception		100%		n/a	0	4	0	11	0	4	0	1
cl.duoc.cordillera.dataservice		100%		n/a	0	2	0	3	0	2	0	1
cl.duoc.cordillera.dataservice.model		100%		n/a	0	1	0	2	0	1	0	1
Total	0 of 467	100%	0 of 2	100%	0	28	0	71	0	27	0	7

Figura 2. Cobertura JaCoCo — Data Service (100% instrucciones y ramas)

Todos los paquetes en verde. Cero instrucciones sin cubrir. Pruebas con `@DataJpaTest` sobre H2 en memoria para `config` y `service`; el `controller` y el manejo de excepciones también tienen cobertura completa.

2.3 KPI Service

kpi-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
cl.duoc.cordillera.kpiservice.config		4%		0%	7	10	56	59	5	8	2	4
cl.duoc.cordillera.kpiservice.exception		34%		50%	4	6	10	13	3	5	0	1
cl.duoc.cordillera.kpiservice		37%		n/a	1	2	2	3	1	2	0	1
cl.duoc.cordillera.kpiservice.service.calculator		100%		100%	0	19	0	31	0	14	0	5
cl.duoc.cordillera.kpiservice.service		100%		n/a	0	9	0	25	0	9	0	1
cl.duoc.cordillera.kpiservice.controller		100%		n/a	0	7	0	10	0	7	0	1
cl.duoc.cordillera.kpiservice.model		100%		n/a	0	13	0	13	0	13	0	1
cl.duoc.cordillera.kpiservice.client		100%		100%	0	5	0	9	0	4	0	2
Total	255 of 657	61%	5 of 18	72%	12	71	68	163	9	62	2	16

Figura 3. Cobertura JaCoCo — KPI Service (61% instrucciones, 72% ramas)

Los paquetes de negocio están bien cubiertos: *calculator*, *service*, *controller*, *model* y *client* llegan al 100%. El paquete *config* (4%) arrastra el promedio hacia abajo porque contiene beans de Spring que no se instancian en pruebas unitarias estándar.

2.4 Report Service

report-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
cl.duoc.cordillera.reportservice.config		7%		0%	3	6	33	36	2	5	1	3
cl.duoc.cordillera.reportservice.service		79%		63%	11	28	10	58	1	13	0	1
cl.duoc.cordillera.reportservice.exception		41%		50%	3	5	8	11	2	4	0	1
cl.duoc.cordillera.reportservice.service.exportador		98%		90%	3	48	2	171	0	32	0	4
cl.duoc.cordillera.reportservice		37%		n/a	1	2	2	3	1	2	0	1
cl.duoc.cordillera.reportservice.controller		97%		80%	2	16	1	28	0	10	0	1
cl.duoc.cordillera.reportservice.service.client		100%		100%	0	6	0	10	0	4	0	1
cl.duoc.cordillera.reportservice.model		100%		80%	2	6	0	7	0	1	0	1
Total	226 of 1.652	86%	21 of 88	76%	25	117	56	324	6	71	1	13

Figura 4. Cobertura JaCoCo — Report Service (86% instrucciones, 76% ramas)

El paquete *service.exportador* alcanza 98% de instrucciones y 90% de ramas, que es donde está la lógica más compleja. El *service.client* llega al 100%. Lo que queda sin cubrir está principalmente en *config* (7%) y en el manejo de excepciones (41%).

3. Patrones de Diseño en las Pruebas

Para aislar las capas sin levantar el contexto completo de Spring aplicamos tres patrones:

Patrón	Componentes	Descripción
Repository Pattern	Data Service	Usa <code>@DataJpaTest</code> con H2 en memoria para desacoplar la capa de acceso a datos. Los servicios se prueban con mocks, sin necesidad de base de datos real.
Factory Method	KpiFactory, ExportadorFactory	Permite verificar que se selecciona la estrategia correcta: <code>verify().obtenerCalculador()</code> confirma exactamente una invocación por solicitud.
Circuit Breaker	BFF Gateway, Report Service	Prueba escenarios de fallo controlado (timeout, error 500) y verifica que el sistema responde con status Degradado en lugar de propagar el error.

4. Casos de Prueba Críticos

Siete casos que cubren los flujos principales: persistencia, manejo de errores, idempotencia y resiliencia ante caídas de servicios downstream.

ID	Componente	Escenario	Resultado
CA-001	Data Service	crear_conPayloadValido_debePersistirYRetornar	PASS
CA-002	Data Service	actualizar_conIdInexistente_debeLanzarNoSuchElementException (404)	PASS
CA-003	KPI Service	obtenerCalculador_categoriaInvalida_lanzaIllegalArgumentException	PASS
CA-004	KPI Service	create_invocaObtenerCalculador_exactamente1Vez	PASS
CA-005	Report Service	generarReporte_conMismoPeriodo_retornaExistente (Idempotencia)	PASS
CA-006	BFF Gateway	getDashboard_conKpiServiceCaído_retornaStatusDegradado (Resiliencia)	PASS
CA-007	Auth Service	autenticar_conCredencialesValidas_retornaAuthResponse	PASS

5. Conclusiones

Los cuatro servicios pasan el quality gate. El Data Service llega al 100%; los otros tres están entre 61% y 88% de instrucciones, con los paquetes de negocio bien cubiertos en todos los casos.

Los siete casos críticos pasan sin errores. Incluyen el escenario de Circuit Breaker (CA-006), que es el más importante para verificar que el BFF no se rompe cuando el KPI Service cae.

El mayor riesgo de cobertura está en los paquetes de configuración e infraestructura, que son difíciles de alcanzar con pruebas unitarias. Una opción es excluirlos explícitamente del quality gate usando la anotación `@ExcludeFromJacocoGeneratedReport` o filtros en el pom.xml, para que el porcentaje refleje solo el código de negocio.